

q1_result_analyzer.py - C:/Users/harsa/q1_result_analyzer.py (3.14.3)

File Edit Format Run Options Window Help

```
def analyze_result(name, roll, marks):
    total = sum(marks)
    average = total / len(marks)

    if average >= 90:
        grade = "A"
    elif average >= 75:
        grade = "B"
    elif average >= 60:
        grade = "C"
    elif average >= 40:
        grade = "D"
    else:
        grade = "Fail"

    print(f"Student: {name} (Roll: {roll})")
    print(f"Total: {total}, Average: {average}")
    print(f"Grade: {grade}")

    below_40 = []

    for i in range(len(marks)):
        if marks[i] < 40:
            below_40.append(f"Subject {i + 1}")

    if below_40:
        print("Subjects below 40:", ", ".join(below_40))
    else:
        print("No subjects below 40")

name = "Aarav"
roll = 101
marks = [88.5, 35.0, 76.0, 92.5, 48.0]

analyze_result(name, roll, marks)
```

IDLE Shell 3.14.3

File Edit Shell Debug Options Window Help

Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32

Enter "help" below or click "Help" above for more information.

>>>

===== RESTART: C:/Users/harsa/q1_result_analyzer.py =====

Student: Aarav (Roll: 101)

Total: 340.0, Average: 68.0

Grade: C

Subjects below 40: Subject 2

>>>

```
def add_book(catalog, book_id, title, author, year):
    catalog[book_id] = (title, author, year)

def borrow_book(catalog, borrowed_books, book_id):
    if book_id in catalog:
        if book_id not in borrowed_books:
            borrowed_books.append(book_id)
            print(f"Book {book_id} borrowed successfully.")
        else:
            print("Book already borrowed.")
    else:
        print("Book not found.")

def return_book(borrowed_books, book_id):
    if book_id in borrowed_books:
        borrowed_books.remove(book_id)
        print(f"Book {book_id} returned successfully.")
    else:
        print("Book was not borrowed.")

def register_member(members, member_id):
    members.add(member_id)

def show_available(catalog, borrowed_books):
    print("\nAvailable Books:")
    for book_id, details in catalog.items():
        if book_id not in borrowed_books:
            print(f"{book_id}: {details[0]} by {details[1]} ({details[2]})")

def main():
    catalog = {}
    borrowed_books = []
    members = set()

    add_book(catalog, 101, "Python Basics", "John Smith", 2020)
    add_book(catalog, 102, "Data Structures", "Alice Brown", 2021)
    add_book(catalog, 103, "Machine Learning", "Andrew Ng", 2022)
    add_book(catalog, 104, "Artificial Intelligence", "Russell", 2019)

    register_member(members, 1)
    register_member(members, 2)
    register_member(members, 3)
```

```

def return_book(borrowed_books, book_id):
    if book_id in borrowed_books:
        borrowed_books.remove(book_id)
        print(f"Book {book_id} returned successfully.")
    else:
        print("Book was not borrowed.")

def register_member(members, member_id):
    members.add(member_id)

def show_available(catalog, borrowed_books):
    print("\nAvailable Books:")
    for book_id, details in catalog.items():
        if book_id not in borrowed_books:
            print(f"{book_id}: {details[0]} by {details[1]} ({details[2]})")

def main():
    catalog = {}
    borrowed_books = []
    members = set()

    add_book(catalog, 101, "Python Basics", "John Smith", 2020)
    add_book(catalog, 102, "Data Structures", "Alice Brown", 2021)
    add_book(catalog, 103, "Machine Learning", "Andrew Ng", 2022)
    add_book(catalog, 104, "Artificial Intelligence", "Russell", 2019)

    register_member(members, 1)
    register_member(members, 2)
    register_member(members, 3)
    register_member(members, 2)

    print("Members:", members)

    borrow_book(catalog, borrowed_books, 101)
    borrow_book(catalog, borrowed_books, 103)

    return_book(borrowed_books, 101)

    show_available(catalog, borrowed_books)

if __name__ == "__main__":
    main()

```

```
===== RESTART: C:/Users/narsa/q1_result_analyzer.py ==
```

```
Members: {1, 2, 3}
```

```
Book 101 borrowed successfully.
```

```
Book 103 borrowed successfully.
```

```
Book 101 returned successfully.
```

```
Available Books:
```

```
101: Python Basics by John Smith (2020)
```

```
102: Data Structures by Alice Brown (2021)
```

```
104: Artificial Intelligence by Russell (2019)
```

```
>>>
```

```
def add_item(item, cart=[]):
    cart.append(item)
    return cart

print(add_item("apple"))
print(add_item("banana"))
print(add_item("milk", cart=["bread"]))
print(add_item("eggs"))

def add_item_fixed(item, cart=None):
    if cart is None:
        cart = []

    cart.append(item)
    return cart

print("\nFixed Function:")
print(add_item_fixed("apple"))
print(add_item_fixed("banana"))
print(add_item_fixed("milk", ["bread"]))
print(add_item_fixed("eggs"))

|
def create_cart(owner, discount=0):
    return {
        "owner": owner,
        "items": [],
        "discount": discount
    }

def add_to_cart(cart, name, price, qty=1):
    cart["items"].append({
        "name": name,
        "price": price,
        "qty": qty
    })

def update_price(price_tuple, new_price):
    try:
        price_tuple[1] = new_price
```

```

def create_cart(owner, discount=0):
    return {
        "owner": owner,
        "items": [],
        "discount": discount
    }

def add_to_cart(cart, name, price, qty=1):
    cart["items"].append({
        "name": name,
        "price": price,
        "qty": qty
    })

def update_price(price_tuple, new_price):
    try:
        price_tuple[1] = new_price
    except TypeError as e:
        print("\nTuple Error:")
        print(e)

def calculate_total(cart):
    total = 0

    for item in cart["items"]:
        total += item["price"] * item["qty"]

    discount = total * cart["discount"] / 100
    final_total = total - discount

    return final_total

cart1 = create_cart("Aarav", 10)
cart2 = create_cart("Priya")

add_to_cart(cart1, "Laptop", 50000, 1)
add_to_cart(cart1, "Mouse", 1000, 2)

```

```
File Edit Format View Options Window Help
    "price": price,
    "qty": qty
})
```

```
def update_price(price_tuple, new_price):
    try:
        price_tuple[1] = new_price
    except TypeError as e:
        print("\nTuple Error:")
        print(e)

def calculate_total(cart):
    total = 0
    for item in cart["items"]:
        total += item["price"] * item["qty"]

    discount = total * cart["discount"] / 100
    final_total = total - discount

    return final_total
cart1 = create_cart("Aarav", 10)
cart2 = create_cart("Priya")

add_to_cart(cart1, "Laptop", 50000, 1)
add_to_cart(cart1, "Mouse", 1000, 2)

add_to_cart(cart2, "Book", 600, 3)
add_to_cart(cart2, "Pen", 20, 5)

print("\nCustomer 1 Cart")
print(cart1)

print("\nCustomer 2 Cart")
print(cart2)

print("\nCustomer 1 Total:", calculate_total(cart1))
print("Customer 2 Total:", calculate_total(cart2))

price = ("Laptop", 50000)
update_price(price, 55000)
```

Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32

Enter "help" below or click "Help" above for more information.

>>>

===== RESTART: C:/Users/harsa/q1_result_analyzer.py =====

Members: {1, 2, 3}

Book 101 borrowed successfully.

Book 103 borrowed successfully.

Book 101 returned successfully.

Available Books:

101: Python Basics by John Smith (2020)

102: Data Structures by Alice Brown (2021)

104: Artificial Intelligence by Russell (2019)

>>>

|